

08. Модуль 3. Задание 7. Демо 2026. СиСА

Задание

7. На сервере HQ-SRV реализуйте мониторинг устройств с помощью открытого программного обеспечения

- ▶ Обеспечьте доступность по URL - <http://mon.au-team.irpo>  для сетей офиса HQ, внесите изменения в инфраструктуру разрешения доменных имён
- ▶ Мониторить нужно устройства HQ-SRV и BR-SRV
- ▶ В мониторинге должны визуально отображаться нагрузка на ЦП, объем занятой ОП и основного накопителя
- ▶ Логин и пароль для службы мониторинга admin P@ssw0rd
- ▶ Организуйте доступ к мониторингу для HQ-CLI, без внешнего доступа
- ▶ Выбор программного обеспечения, основание выбора и основные параметры с указанием порта, на котором работает мониторинг, отметьте в отчёте

Реализация

Установка Prometheus на HQ-SRV



Виртуальная машина HQ-SRV

Устанавливаем `prometheus`

```
1 | # dnf install golang-github-prometheus -y
```

Запускаем `prometheus` и добавляем его в автозагрузку

```
1 | # systemctl enable prometheus --now
```

Проверяем работу Prometheus введя в браузере на HQ-CLI IP адрес HQ-SRV и порт 9090

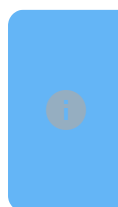
Можно проверить, что сбор данных для сервиса prometheus работает. Меню **Status -> Targets**. Должна отображаться одна цель **prometheus**

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	7.834s ago	8.174ms	

Установка и настройка Node Exporter на HQ-SRV и BR-SRV



Виртуальная машина HQ-SRV



Используем один из самых популярных экспортеров – **Node Exporter**, который собирает множество системных метрик (процессор, память и т.д.) для оценки состояния сервера и его нагрузки. **Node Exporter** доступен в репозитории РЕД ОС

Создаем пользователя **nodeusr**

```
1 | # useradd -rs /bin/false nodeusr
```

Устанавливаем экспортер

```
1 | # dnf install prometheus-node_exporter -y
```

Запускаем `node_exporter` и добавляем его в автозагрузку

```
1 | # systemctl enable node_exporter --now
```



Добавление `node_exporter` в `prometheus`

На `HQ-SRV` открываем конфигурационный файл `prometheus`

```
1 | # nano /etc/prometheus/prometheus.yml
```

Дополняем секцию `scrape_configs` следующими строками

```
1 | - job_name: 'HQ-SRV'
2 |   static_configs:
3 |     - targets: ['192.168.100.2:9100']
```

где

`192.168.100.2` - ip адресс HQ-SRV

Перезапускаем `prometheus`

```
1 | # systemctl restart prometheus
```



Виртуальная машина `BR-SRV`

Создаем пользователя `nodeusr`

```
1 | # useradd -rs /bin/false nodeusr
```

Устанавливаем экспортер

```
1 | # dnf install prometheus-node_exporter -y
```

Запускаем `node_exporter` и добавляем его в автозагрузку

```
1 | # systemctl enable node_exporter --now
```



Добавление node_exporter в [prometheus](#)

На [HQ-SRV](#) открываем конфигурационный файл [prometheus](#)

```
1 | # nano /etc/prometheus/prometheus.yml
```

Дополняем секцию [scrape_configs](#) следующими строками

```
1 | - job_name: 'BR-SRV'
2 |   static_configs:
3 |     - targets: ['192.168.20.2:9100']
```

где

[192.168.20.2](#) - ip адресс BR-SRV

Перезапускаем [prometheus](#)

```
1 | # systemctl restart prometheus
```

Получаем

```
GNU nano 8.0 /etc/prometheus/prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: 'prometheus'

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'HQ-SRV'
    static_configs:
      - targets: ['192.168.100.2:9100']

  - job_name: 'BR-SRV'
    static_configs:
      - targets: ['192.168.20.2:9100']
```

Проверка

Проверяем что в [Prometheus](#) данные передаются

На [HQ-CLI](#) в браузере вводим IP [HQ-SRV](#) и порт [9090](#)

1 Prometheus Time Series: X +

192.168.100.2:9090 Prometheus Time Series Collection and Processing Server

перевести

Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy

BR-SRV (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.20.2:9100/metrics	UP	instance="192.168.20.2:9100" job="BR-SRV" ▾	1.269s ago	18.163ms	

HQ-SRV (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.100.2:9100/metrics	UP	instance="192.168.100.2:9100" job="HQ-SRV" ▾	6.21s ago	17.747ms	

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus" ▾	3.839s ago	7.316ms	

Установка и настройка Grafana



Виртуальная машина HQ-SRV

Установка Grafana

```
1 | # dnf install grafana -y
```

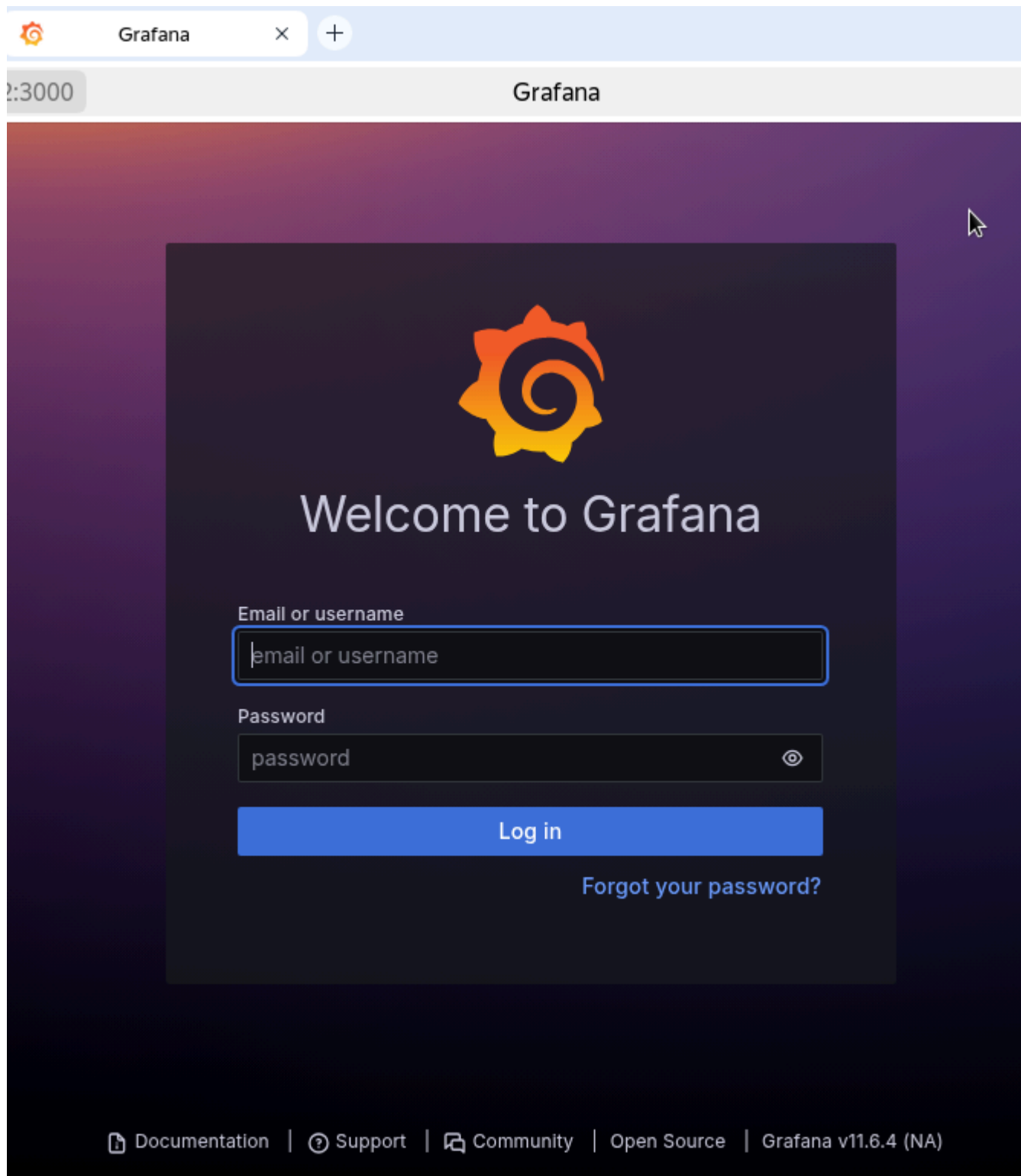
Запускаем `grafana-server` и добавляем его в автозагрузку

```
1 | # systemctl enable grafana-server --now
```

Настройка Grafana



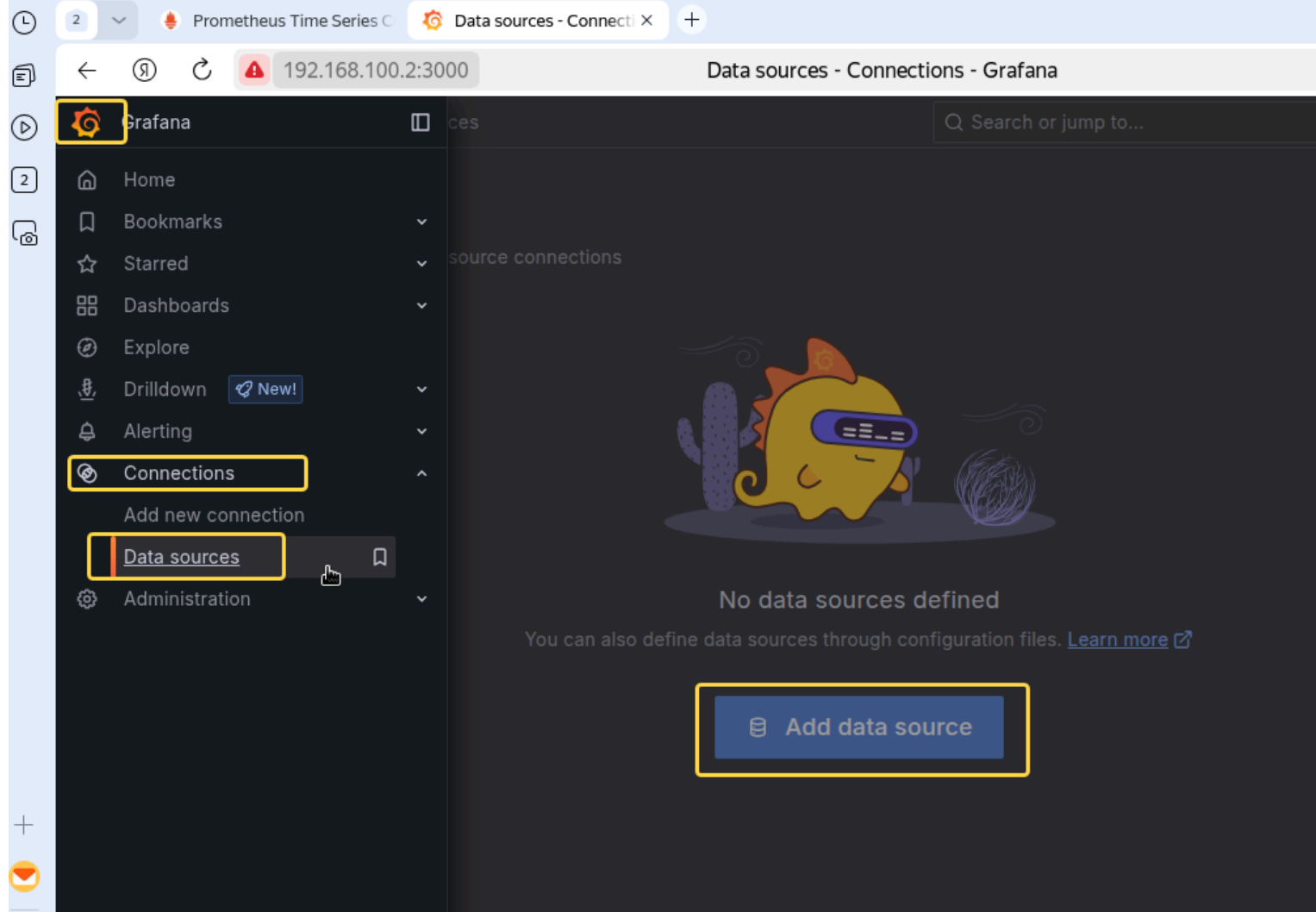
Веб-интерфейс `Grafana` доступен по адресу `http://<IP-адрес_HQ-SRV>:3000`



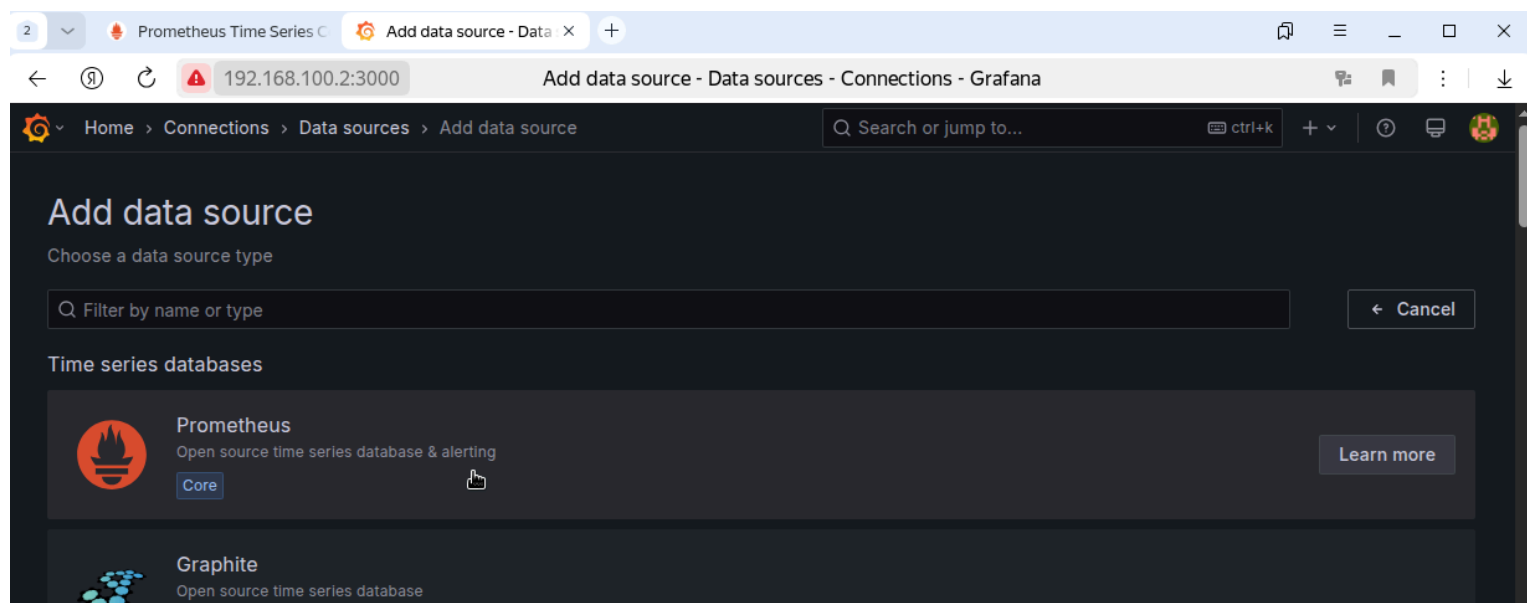
Логин и пароль для первого входа: `admin/admin` . При первом входе будет запрошена смена пароля – меняем согласно заданию на `admin / P@ssw0rd`

“

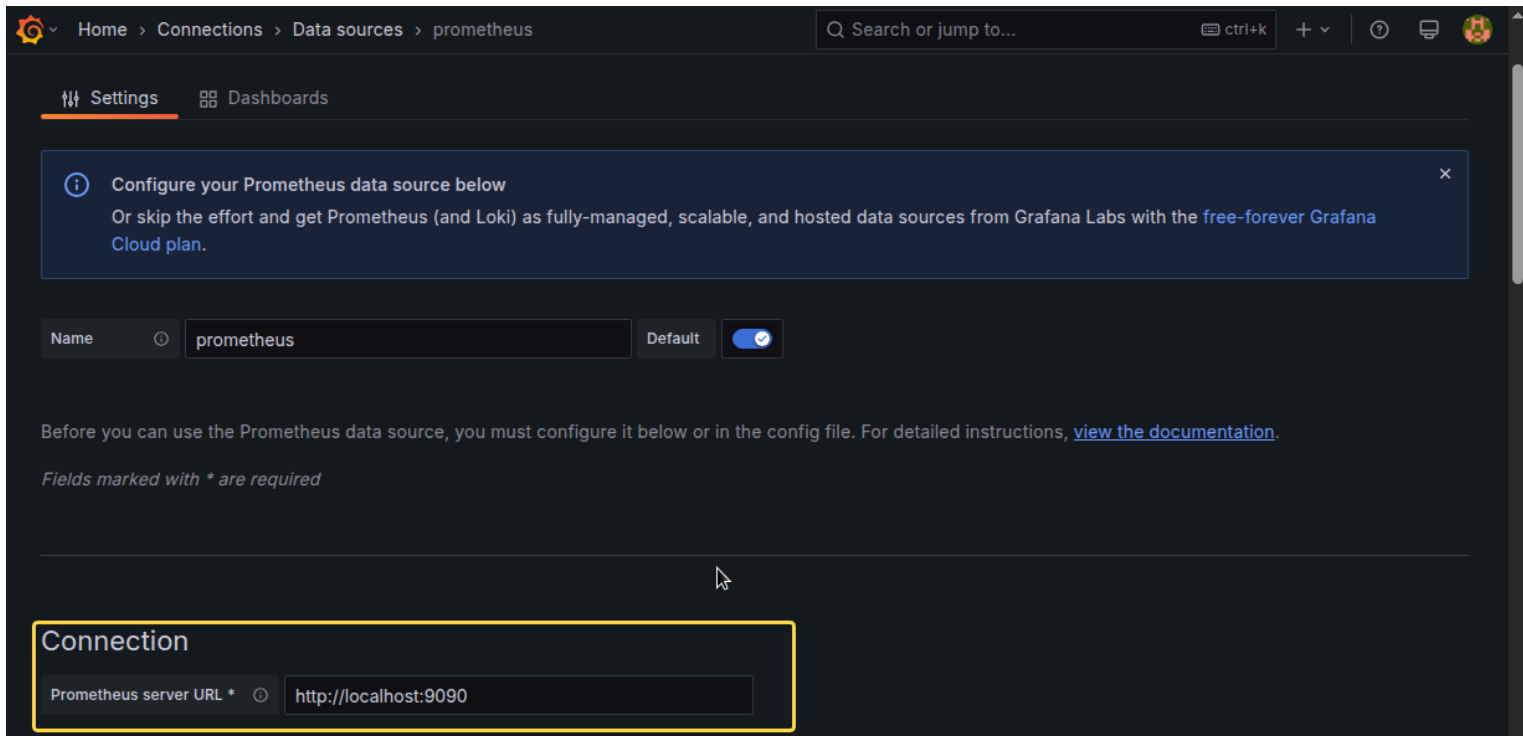
Добавляем источник данных для Grafana, `Home -> Connections -> Data Sources -> Add data source`



Выбираем Prometheus



В настройках указываем IP-адрес и порт сервера с Prometheus (HQ-SRV), остальные настройки оставляем по умолчанию (остальные по желанию)



Так как *Grafana* и *Prometheus* находится на HQ-SRV можно использовать *localhost* , а порт по умолчанию *9090*

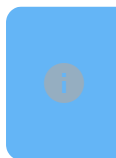


Для сохранения настроек в низу страницы жмем **Save & Test**

Загрузка дашбордов



Загрузка дашбордов



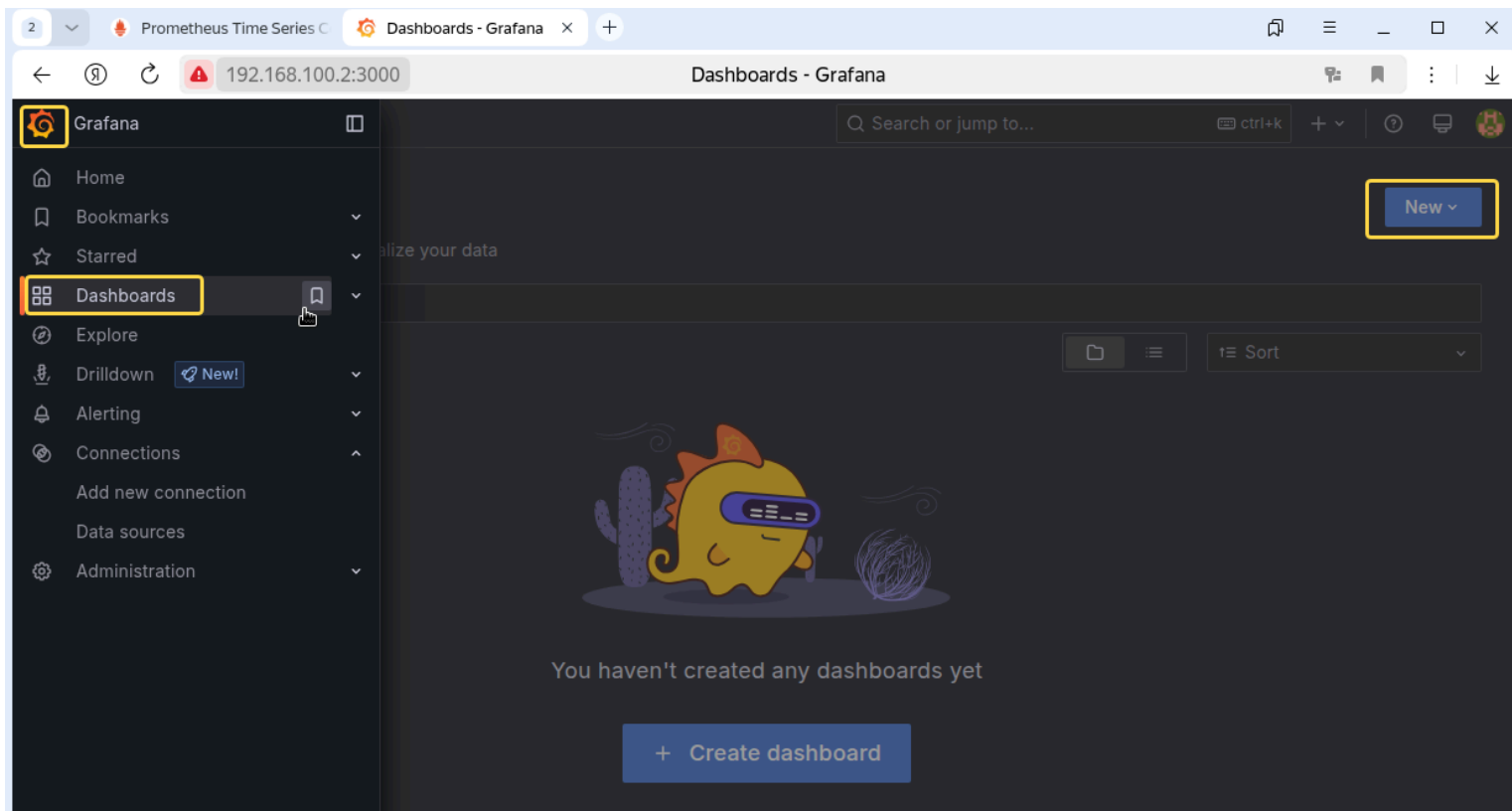
Наиболее универсальным является `id=1860` , его можно [скачать по ссылке](#)  (используем ревизию 27 и ниже)



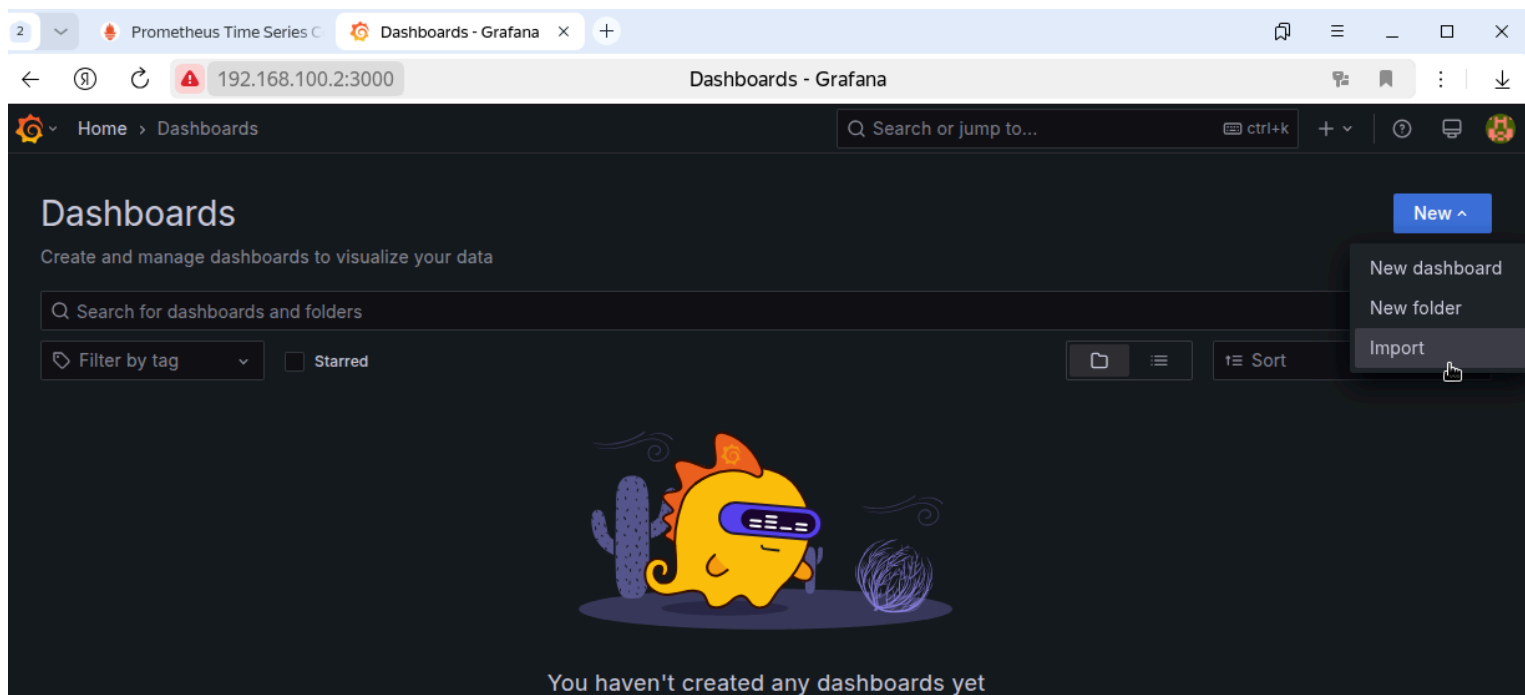
Node Exporter Full | Grafana Labs

[illegible]

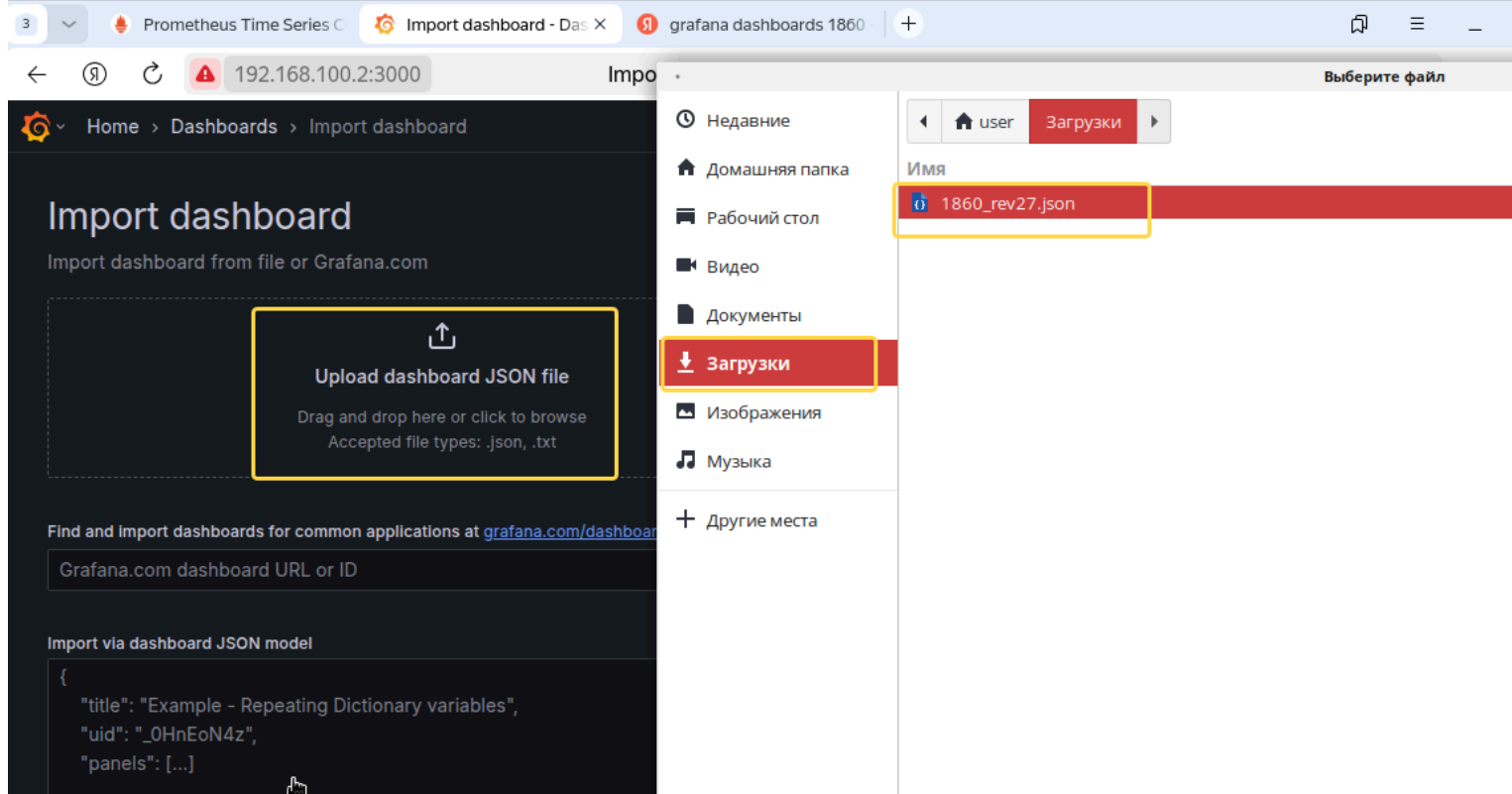
Для импорта дашборда выбираем [Home - Dashboards](#)



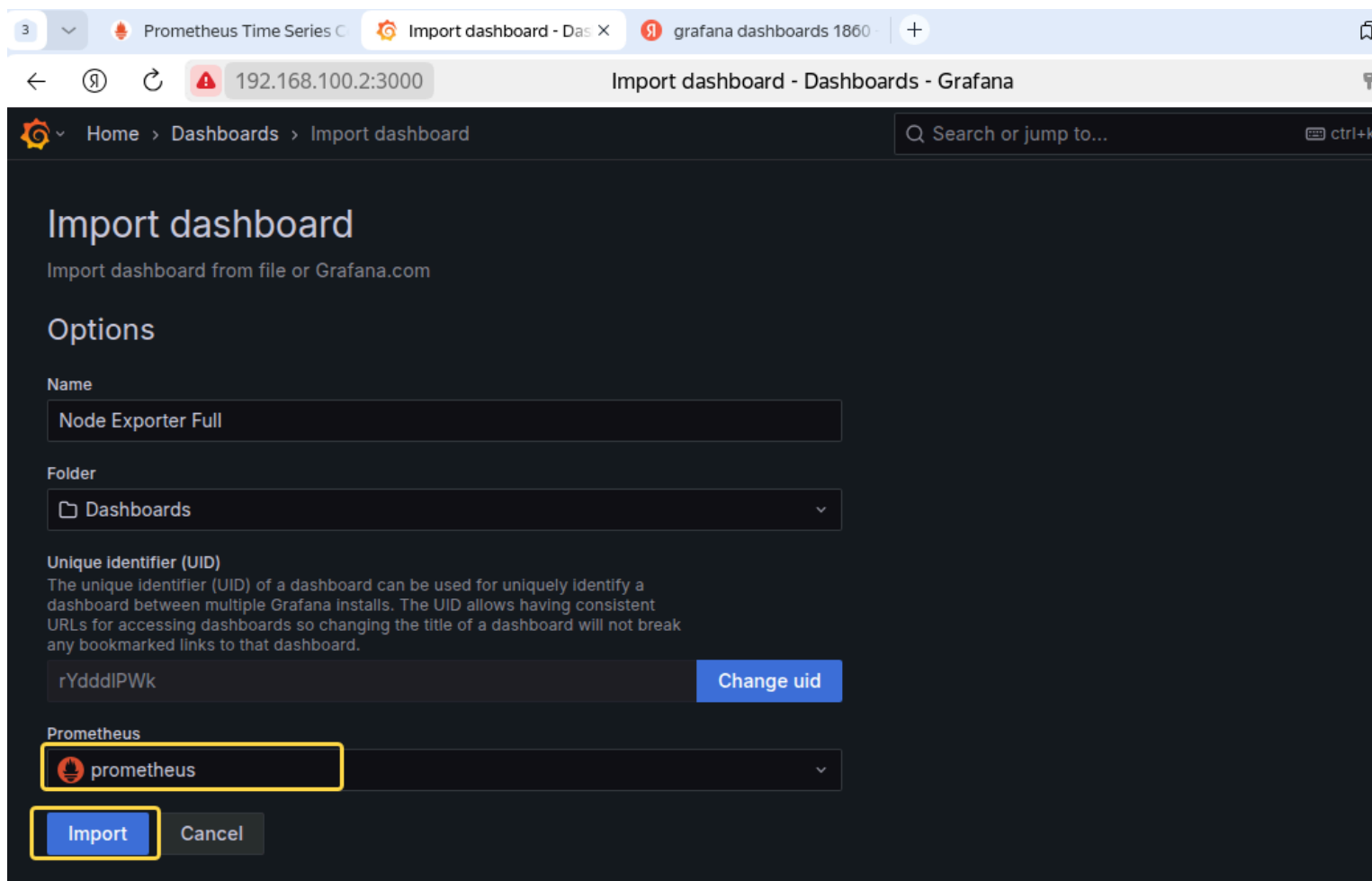
Жмем New - Import



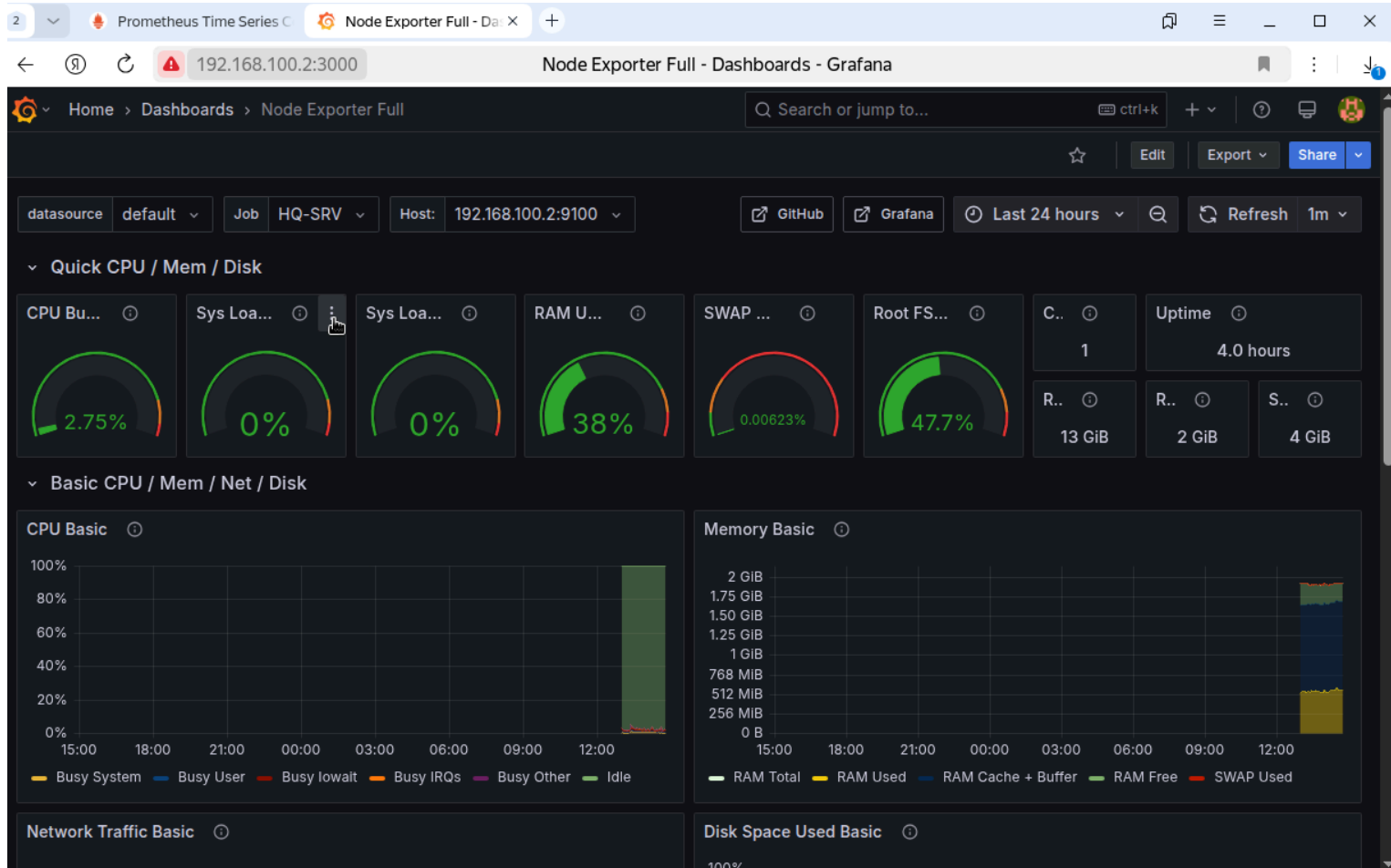
Нажимаем Upload dashboard JSON file , выбираем скачанный ранее JSON-файл



В появившемся окне **Options** выбрать **Prometheus** и нажать **Import**



Откроется загруженный дашборд 1860



Доступность по URL

-  **Виртуальная машина HQ-CLI**
-  **Вариант -1**

В домене SambaDC на HQ-SRV нет в DNS записи `mon.au-team.irpo`. Внесем дополнительные записи в `/etc/hosts` на HQ-CLI

Открываем файл `hosts` на HQ-CLI

```
1 | # nano /etc/hosts
```

Добавляем запись

```
1 | 192.168.100.2    mon.au-team.irpo
```

-  **Вариант -2**

Открываем на редактирование файл зоны `au-team.db` DNS сервера на HQ-SRV

```
1 | # nano /var/named/master/au-team.db
```

И добавляем запись

```
1 | mon IN A 192.168.100.2
```

```
GNU nano 8.0
$TTL 1D
@ IN SOA au-team.irpo. root.au-team.irpo. (
    0 ; serial
    1D ; refresh
    1H ; retry
    1W ; expire
    3H ) ; minimum

IN NS au-team.irpo.
IN A 192.168.100.2
hq-rtr IN A 192.168.100.1
hq-rtr IN A 192.168.200.1
hq-rtr IN A 192.168.99.1
br-rtr IN A 192.168.20.1
hq-srv IN A 192.168.100.2
hq-cli IN A 192.168.200.2
br-srv IN A 192.168.20.2
docker IN A 172.16.1.1
web IN A 172.16.2.1
mon IN A 192.168.100.2
```

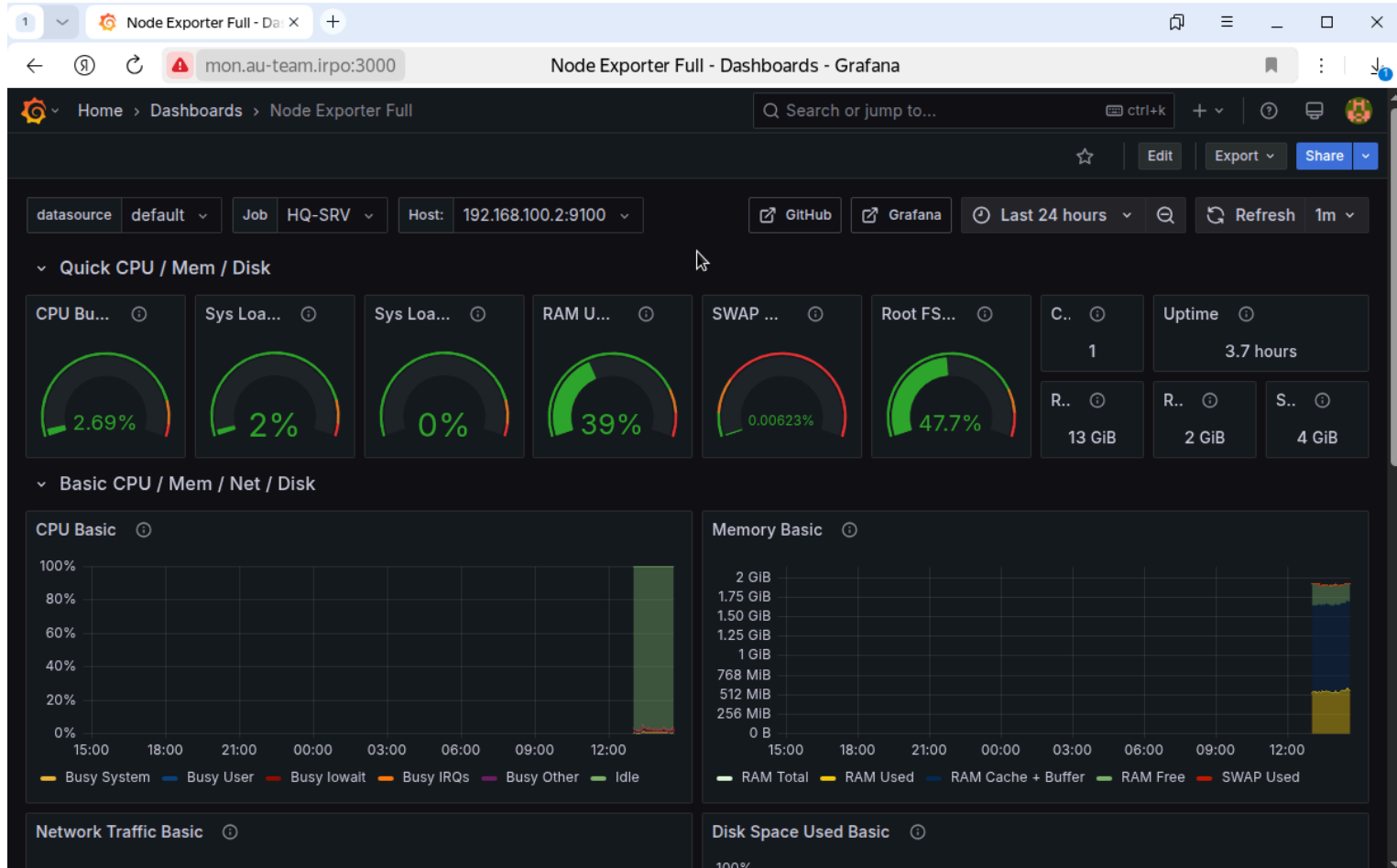
Выполняем перезагрузку файла конфигурации зон службы named

```
1 | # rndc reload
```

Перезагружаем [named](#)

```
1 | # systemctl restart named
```

В браузере на [HQ-CLI](#) вводим <http://mon.au-team.irpo:3000> и попадаем на систему мониторинга [Prometheus](#) с системой визуализации данных [Grafana](#)



В выпадающем списке **Job** можно выбрать сервер для визуализации

